

Data and config management

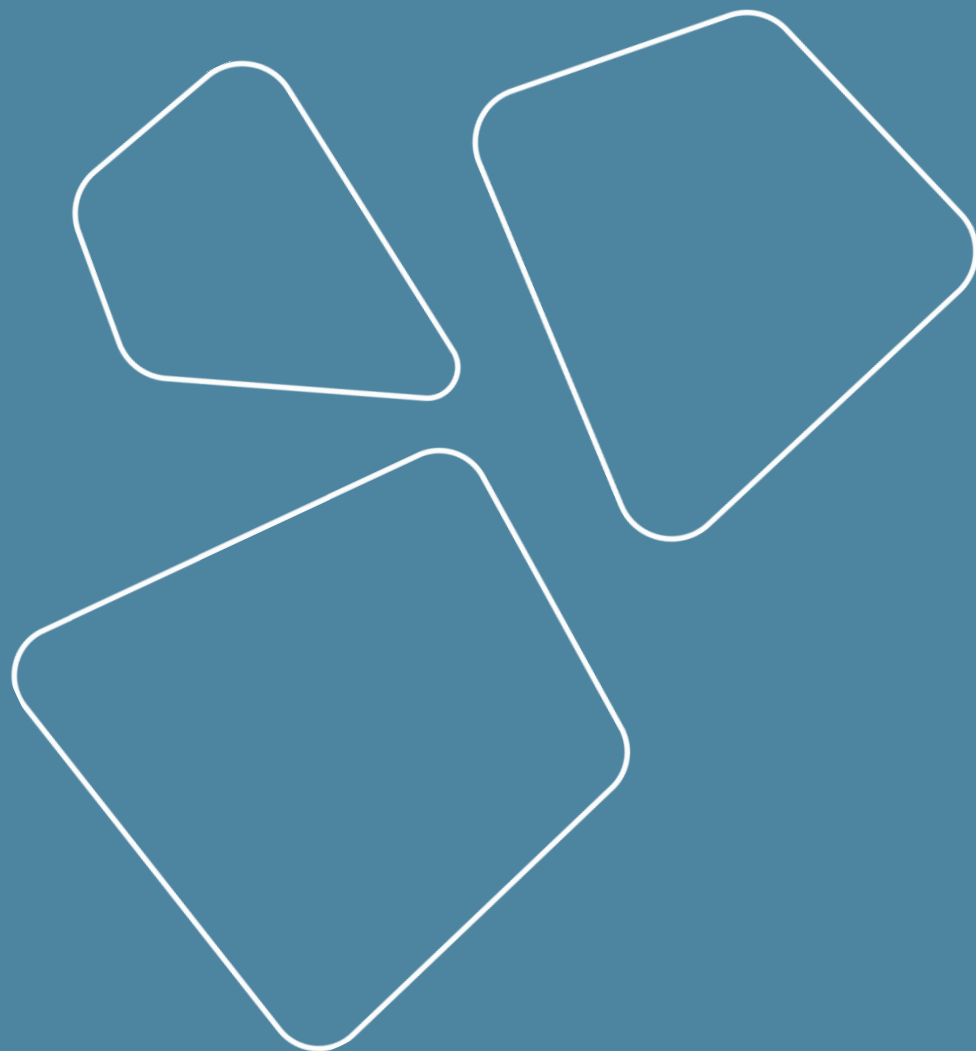
Vladislav Goncharenko



METU, spring 2026

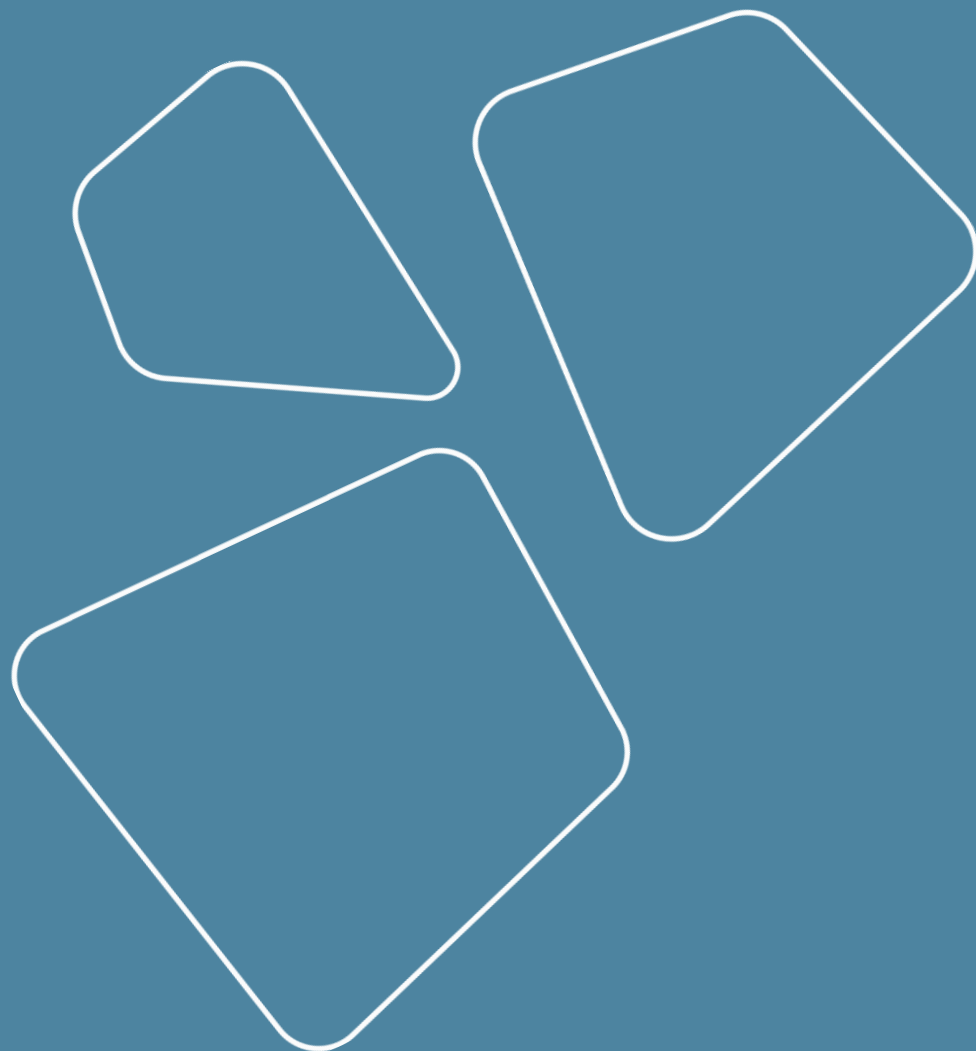
Recap

1. Logging
 - a. MLflow
 - b. ClearML
2. Visualization
3. Frameworks for training
 - a. Lightning
 - b. Hugging face



Outline

1. Data sources
2. Data storage
 - a. small data
 - b. big data
3. Data engineering
 - a. ETL and ELT
 - b. datahub
4. Experiments parametrization
 - a. Hydra
 - b. Vault



Data sources

girafe
ai

01

Data sources

- Primary
 - keyboard input
 - web projects
 - typical architecture: kafka - hadoop - airflow - ml
 - see courses on web and big data
 - photo and video cameras
 - self driving cars
 - many other sensors
 - temperature, humidity, illumination
- Secondary
 - results of sql queries
 - speech to text
 - web scraping
 - LLM
 - synthetic data
 - 3d modelling e.g. <https://carla.org/>
 - augmentation
 - etc...

Characteristics of sources

Factor	Primary Data	Secondary Data
Definition	Primary Data refers to the first-hand data collected by the team. It is collected based on the researcher's needs.	Secondary Data has been collected by other teams in the past. It does not necessarily need to be aligned with the researcher's requirements.
Data	Real-time Data	Historical Data
Process	Time Consuming	Quick and Easy
Cost	Expensive	Economical
Collection Time	Long	Short
Available In	Raw and Crude form	Refined form
Accuracy and Reliability	Very high	Relatively less
Examples	Personal Interviews, Surveys, Observations, etc.	Websites, Articles, Research Papers, Historical Data, etc.

Types of data by structure

- Structured
 - aka externally structured
 - tabular or relational
- Unstructured (internally structured)
 - Texts
 - Images
 - Video
 - Voice
 - Graphs
 -

This naming is a bit weird but this is how it works now.

Kinds of data

Structured Data	Unstructured Data
Structured Data is organized and has a fixed and defined schema.	Unstructured Data does not have any pre-defined structure to it.
Structured data is quantitative data that consists of numbers and values .	Unstructured data is qualitative data that consists of text, images, audio, video , etc.
Structured Data is stored in Relational Databases and Data warehouses .	Unstructured Data generally is stored in Data Lakes .
Structured Data is stored in tabular formats such as SQL databases .	It is typically stored in NoSQL databases .
It is collected from sensors, network logs, web server logs, OLTP systems, etc.	It is sourced from email messages, word-processing documents, pdf files, etc.
It requires less storage and is highly scalable .	It needs more storage space and is difficult to scale .

Data storages

girafe
ai

02

Why not to store in VCS?

- data changes frequently
- data in repo should be pulled in full size
- previous commits also are stored and pulled by default
- git is not meant to be used for binary data
 - it treats files as sequences of lines

1) size

2) ease of access
- analytics
- preproc

3) format
VCS works with
text data



CRUD - main database operations

- Create
- Read
- Update
- Delete

https://en.wikipedia.org/wiki/Create,_read,_update_and_delete

Ways to store data

Relevant both for datasets and trained model artifacts

- Locally
- Remotely
- Distributed
 - small data
 - DVC
 - git LFS
 - lakeFS
 - big data
 - MapReduce paradigm
 - e.g. hadoop stack or YTsaurus
 - S3 based data lakes



Hashing

To track changes in binary files you just need to compare hashes of existing vs new files.

What a simple hash function could be?

Polynomial rolling hash

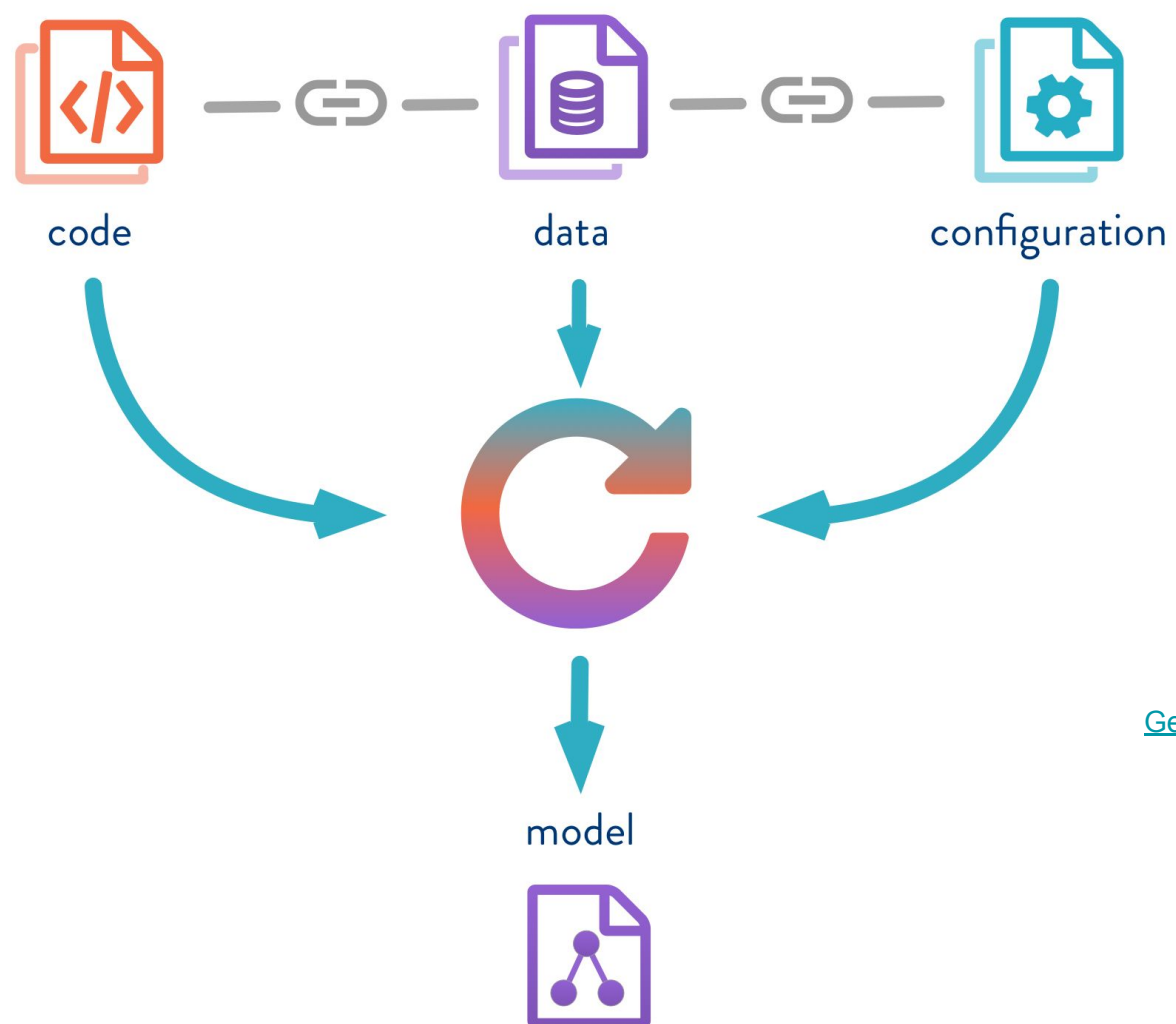
$$H = c_1 a^{k-1} + c_2 a^{k-2} + c_3 a^{k-3} + \dots + c_k a^0$$

How to compute a modern hash?

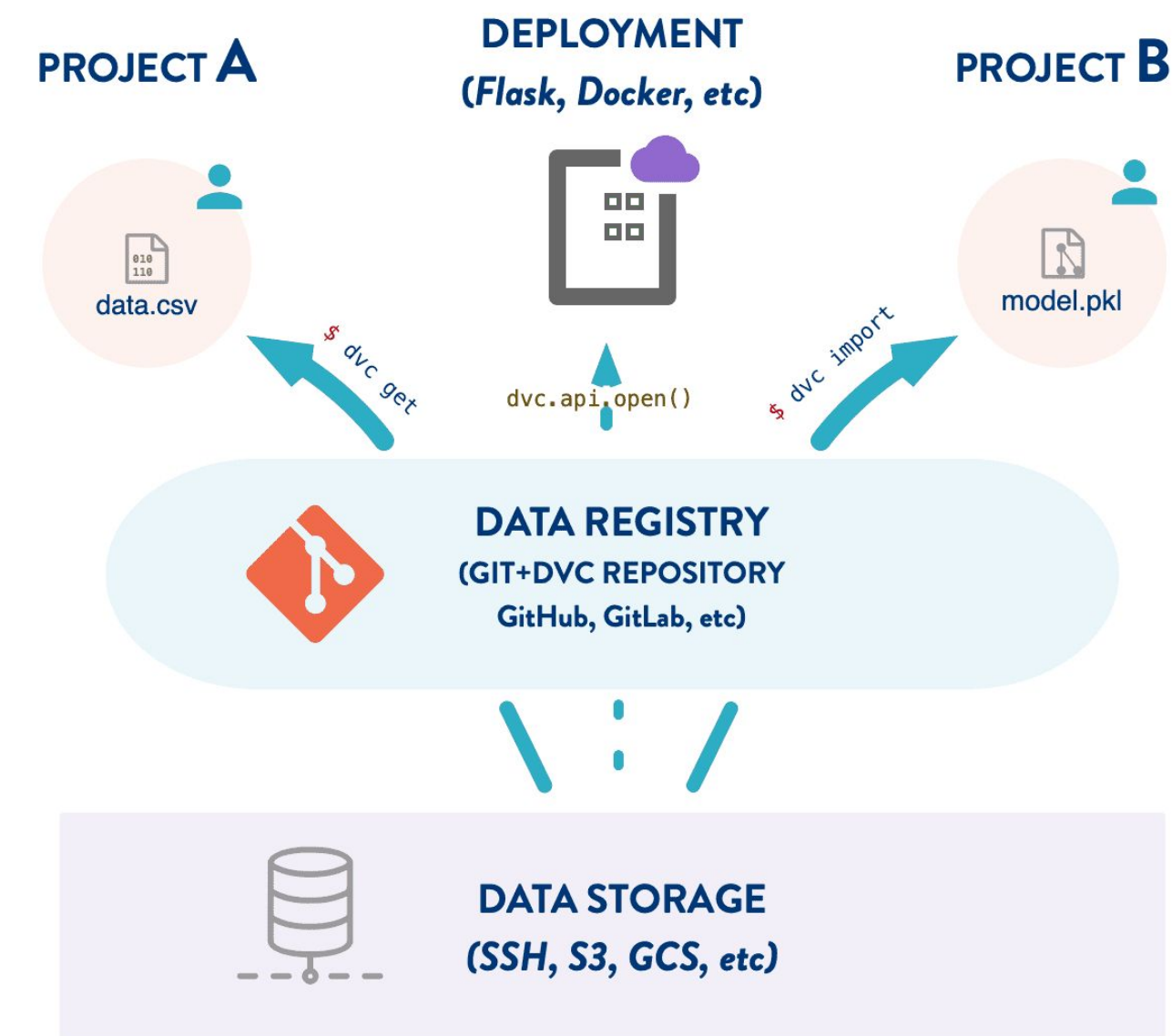
see https://www.youtube.com/watch?v=R_mOWu3s6y4

Data Version Control (DVC)

- git for data is [DVC](#)
- tutorials: [one](#), [two](#)
- [backends](#)
- [Python API](#)



[Get Started with DVC | Data Version Control](#)

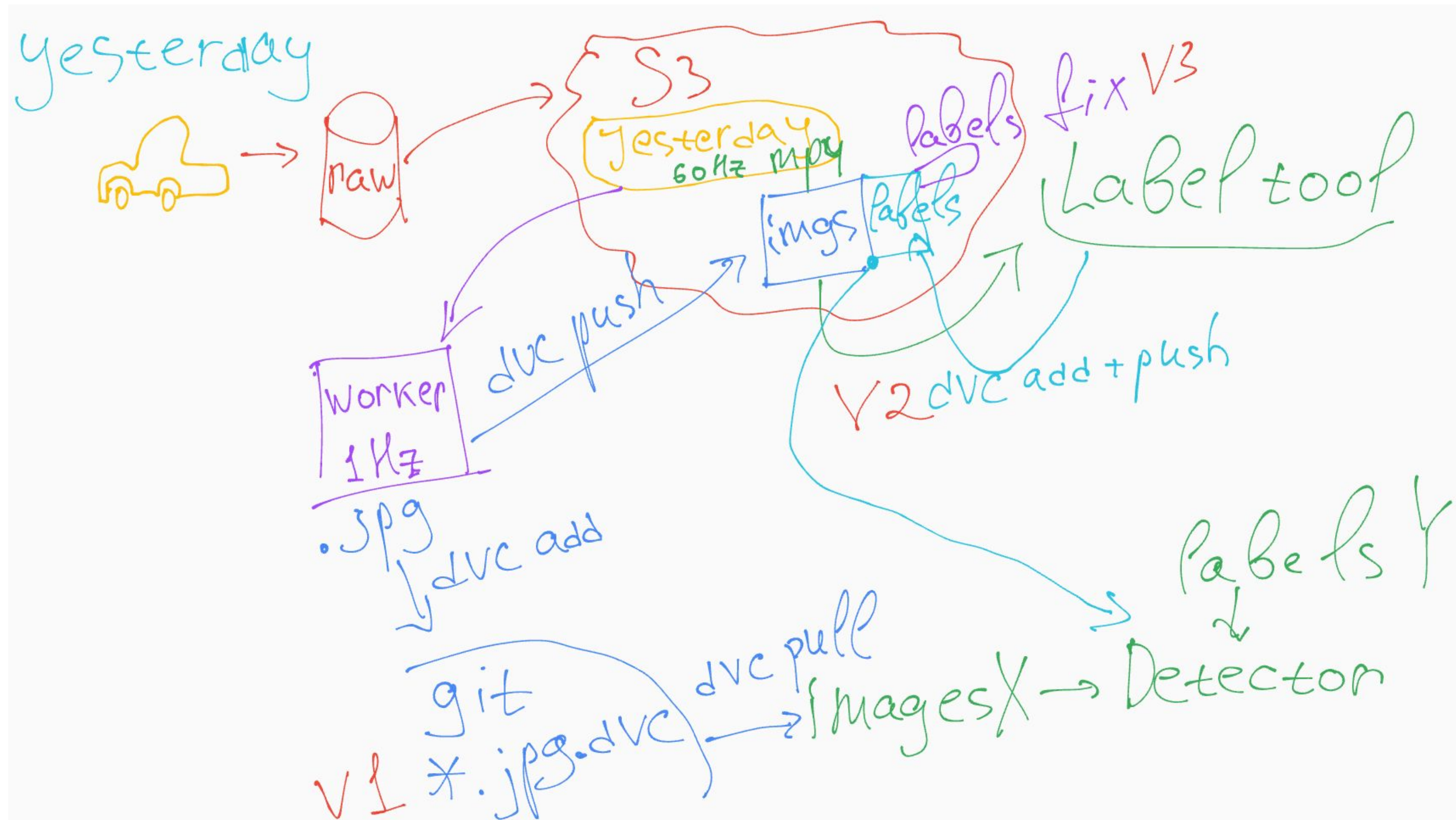


[Get Started with DVC | Data Version Control](#)

WebDAV with Yandex

- <https://dvc.org/doc/user-guide/data-management/remote-storage/webdav>
- <https://yandex.ru/dev/disk/webdav/>
- <https://yandex.ru/dev/disk/doc/ru/concepts/quickstart>

Self-driving car case



Other data versioning systems

- <https://lakefs.io/>
- <https://www.atlassian.com/git/tutorials/git-lfs>

<https://alternativeto.net/software/dvc/>



lakeFS

Small data vs big data

1. Cost of infrastructure
for small data VPS (ssh), s3 storage or GDrive
for big data - need MapReduce (3 VPS for hadoop at least)
2. Tracking of data state
for small data DVC or git LFS stores full snapshot of data
for big data - you don't care, also it's not possible

Feature storages

- <https://github.com/feast-dev/feast>
- <https://github.com/NVIDIA/aistore>
- <https://github.com/qdrant/qdrant>



FEAST



qdrant

Databases

Nowadays they are so versatile that for every single need we have a dedicated solution.

Short overview of DB internals:

<https://www.youtube.com/watch?v=i-FFVM4cIXQ>

PostgreSQL <https://www.youtube.com/watch?v=P8rrhZTPEAQ>

Data Engineering

girafe
ai

03

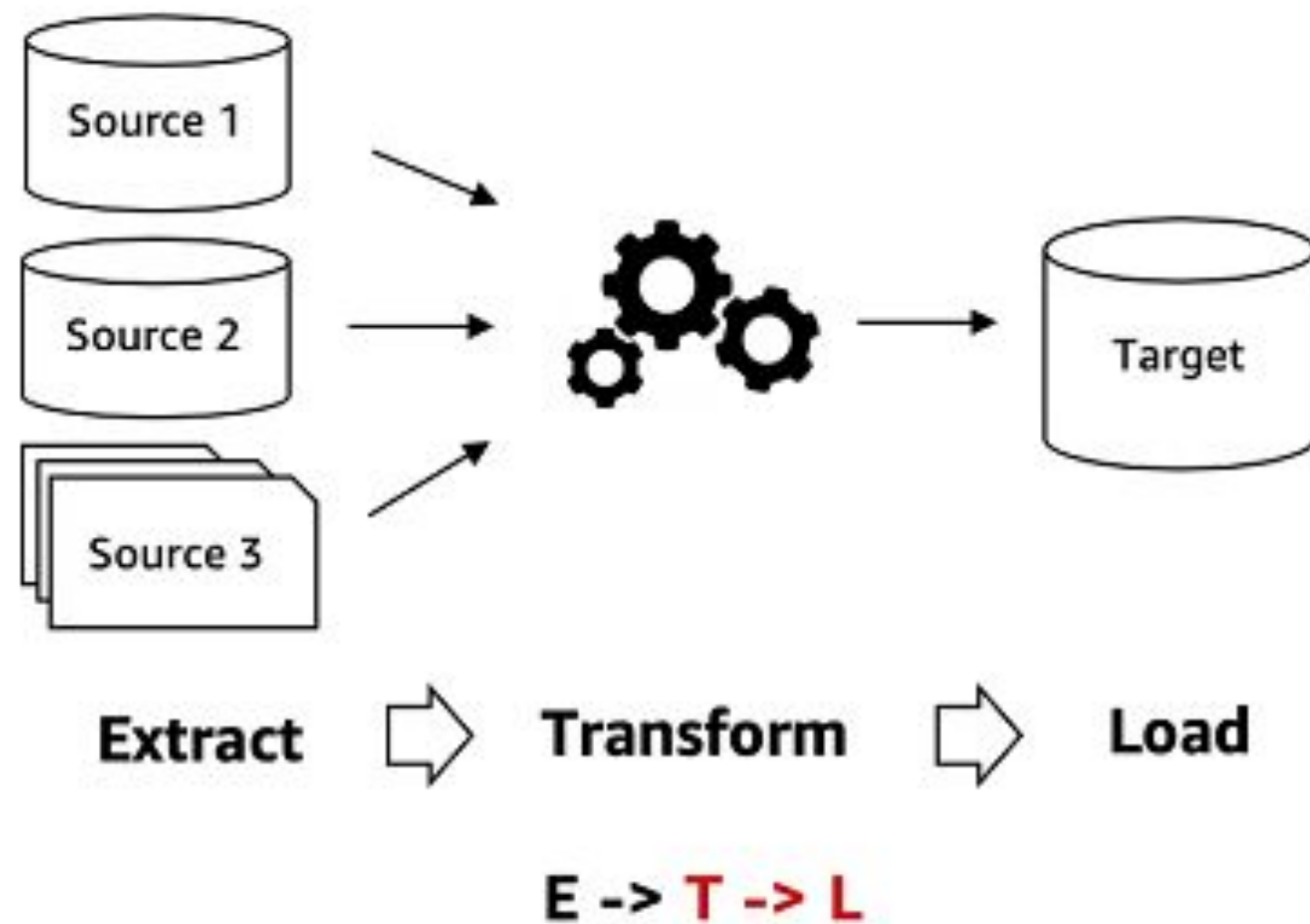
Great course on DE

<https://github.com/DataTalksClub/data-engineering-zoomcamp>

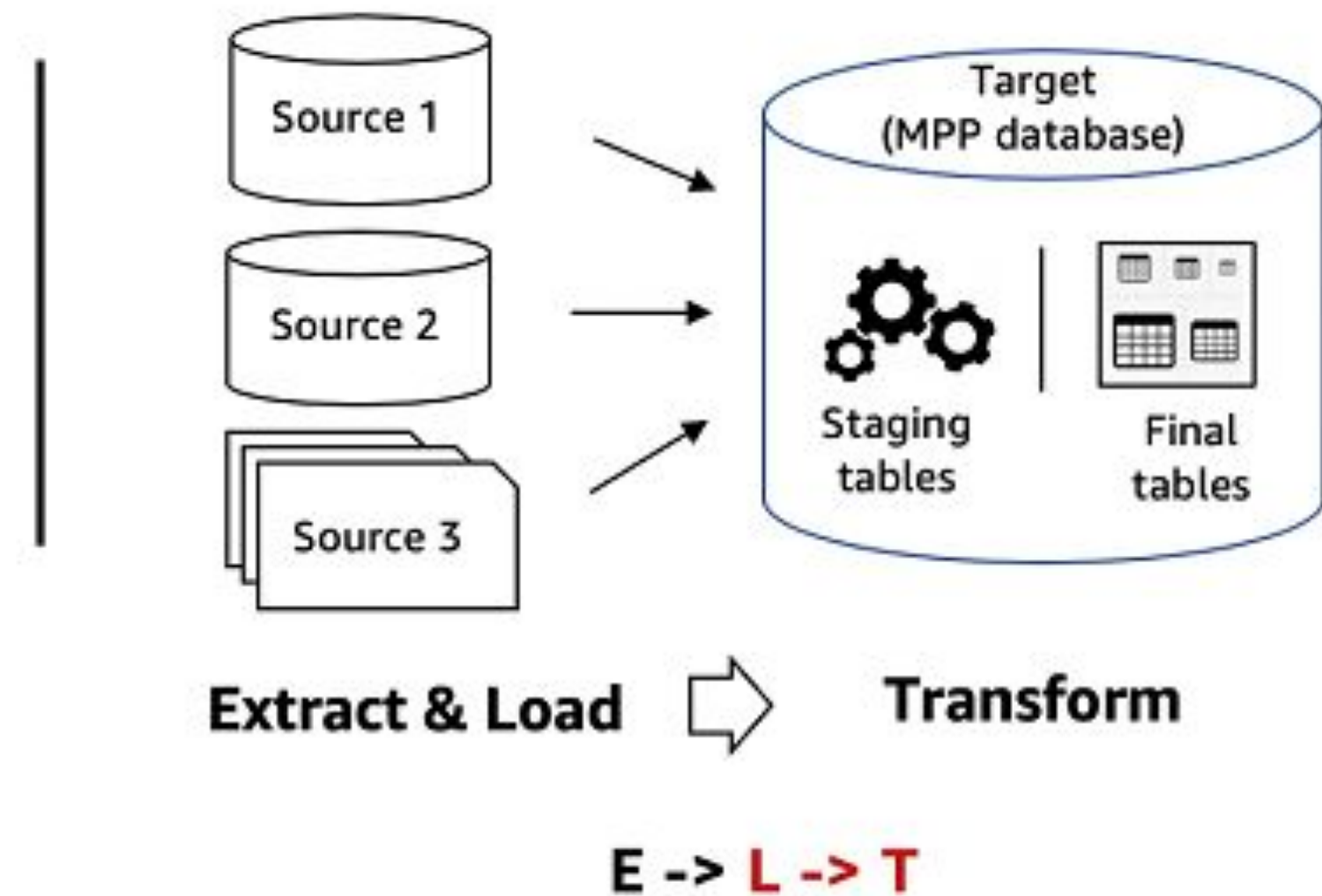
ETL and ELT

ETL: extract, transform, load

ELT: extract, load, transform



1. Site logs
 - a. ELT
2. Site monitoring
 - a. ETL



ETL vs ELT

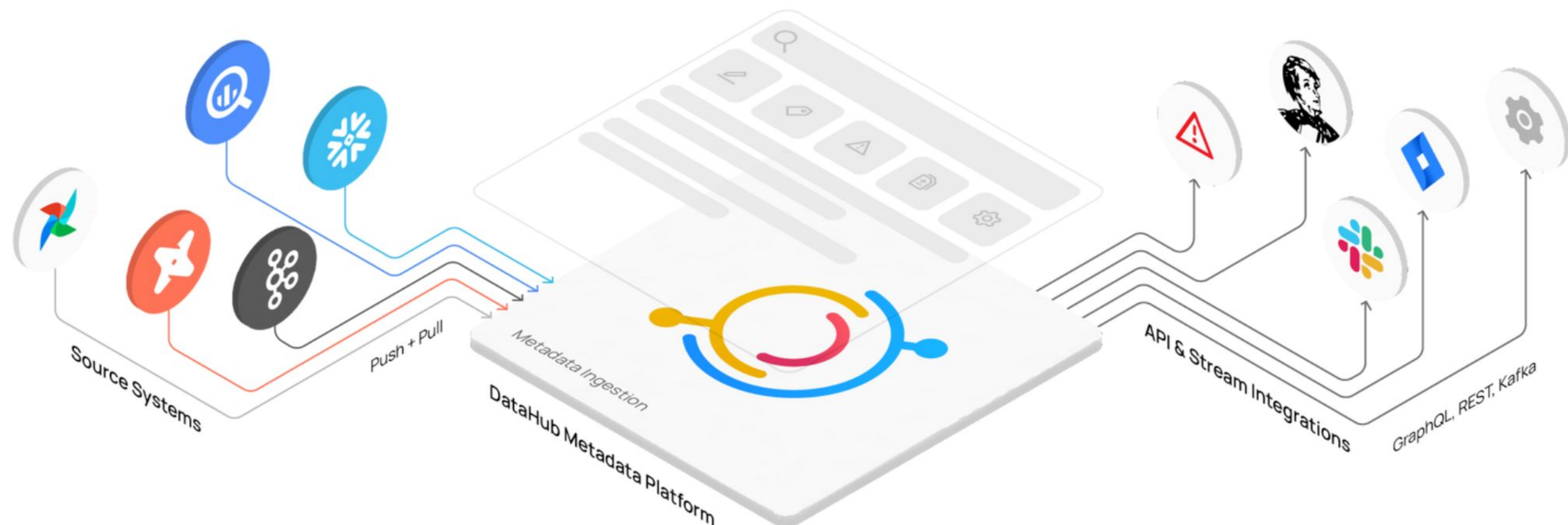
ELT	ETL
ELT tools do not require additional hardware	ETL tools require specific hardware with their own engines to perform transformations
Mostly Hadoop or NoSQL database to store data. Rarely RDBMS is used	RDBMS is used exclusively to store data
As all components are in one system, loading is done only once	As ETL uses staging area, extra time is required to load the data
Time to transform data is independent of the size of data	The system has to wait for large sizes of data. As the size of data increases, transformation time also increases
It is cost effective and available to all business using SaaS solution	Not cost effective for small and medium business
The data transformed is used by data scientists and advanced analysts	The data transformed is used by users reading report and SQL coders
Creates ad hoc views. Low cost for building and maintaining	Views are created based on multiple scripts. Deleting view means deleting data
Best for unstructured and non-relational data. Ideal for data lakes. Suited for very large amounts of data	Best for relational and structured data. Better for small to medium amounts of data

Data Hub

Single place to store meta information for all data

<https://datahubproject.io/>

Kafka^{int}
Redis
user id
item id
Postgres
Str
MongoDB
.....



Database modelling

e.g. [Data vault modeling](#)

Experiments parametrization

girafe
ai

04

Written and run model - what's next?

- Produce the best model in one run - unrealistic
- It requires a large number of experiments
- How not to get confused?
- What if there are several people?
- And then there's rolling into production...



Possible solutions

- Copy and paste - already passed level
- Making git branches for each experiment is impractical
- Use configuration files!



Configuration management systems

- <https://docs.pydantic.dev/latest/>
- .env file with <https://github.com/theskumar/python-dotenv>
- <https://github.com/HBNetwork/python-decouple>
- <https://github.com/dynaconf/dynaconf>
- <https://github.com/facebookresearch/hydra> - made for ML research
- etc...

Config management for ML



<https://github.com/facebookresearch/hydra>

Example of app

- Config is defined by
 - yaml
 - dataclass
- Functions are wrapped to `hydra.main`
- Choose config in CLI
- Saves final version of config
- Example of app refactoring
<https://github.com/ArjanCodes/2021-config/>

```
my_app_type_error.py

from dataclasses import dataclass

import hydra
from hydra.core.config_store import ConfigStore

@dataclass
class MySQLConfig:
    host: str = "localhost"
    port: int = 3306

cs = ConfigStore.instance()
# Registering the Config class with the name 'config'.
cs.store(name="config", node=MySQLConfig)

@hydra.main(version_base=None, config_name="config")
def my_app(cfg: MySQLConfig) -> None:
    # pork should be port!
    if cfg.pork == 80:
        print("Is this a webserver?!")

if __name__ == "__main__":
    my_app()
```

YAML to store values

Supports:

- hierarchical configs
- default values
- variables
- simple functions

Tip:

keep `_self_` at the last line of defaults section by default because it means highest priority.

```
1  defaults:
2    - files: mnist
3    - _self_
4  paths:
5    log: ./runs
6    data: ${hydra:runtime.cwd}/../data/raw
7  params:
8    epoch_count: 20
9    lr: 5e-5
10   batch_size: 128
```


Hierarchical configs

```
@dataclass
class MySQLConfig:
    host: str = "localhost"
    port: int = 3306

@dataclass
class UserInterface:
    title: str = "My app"
    width: int = 1024
    height: int = 768

@dataclass
class MyConfig:
    db: MySQLConfig = field(default_factory=MySQLConfig)
    ui: UserInterface = field(default_factory=UserInterface)

cs = ConfigStore.instance()
cs.store(name="config", node=MyConfig)

@hydra.main(version_base=None, config_name="config")
def my_app(cfg: MyConfig) -> None:
    print(f"Title={cfg.ui.title}, size={cfg.ui.width}x{cfg.ui.height} pixels")
```

Advanced syntax

```
1 defaults:
2   - models/catboost: master
3   - models/optuna: catboost
4   - db_client@client
5   - pools: samples
6   - pools/processors: common_proc
7
8 target: kp
9 validation: online
10 version: 3
11 verbose: true
12
13 predictions: predicted_${.target}_on_${.validation}_v${.version}.csv
14
```


Hydra without main decorator

[The Compose API](#) is useful when `@hydra.main()` is not applicable. For example:

- Inside a Jupyter notebook
- In parts of your application that does not have access to the command line
- If you want to compose multiple configuration objects

```
from hydra.experimental import compose, initialize
from omegaconf import OmegaConf

if __name__ == "__main__":
    # context initialization
    with initialize(config_path="conf", job_name="test_app"):
        cfg = compose(config_name="config", overrides=["db=mysql", "db.user=me"])
        print(OmegaConf.to_yaml(cfg))

    # global initialization
    initialize(config_path="conf", job_name="test_app")
    cfg = compose(config_name="config", overrides=["db=mysql", "db.user=me"])
    print(OmegaConf.to_yaml(cfg))
```

Integrated with popular libraries

- [hydra-zen](#): vanilla wrapper over Hydra
- [lightning-hydra-template](#): template for popular training library
- [hydra-torch](#): wrapper for most popular neural deep learning library

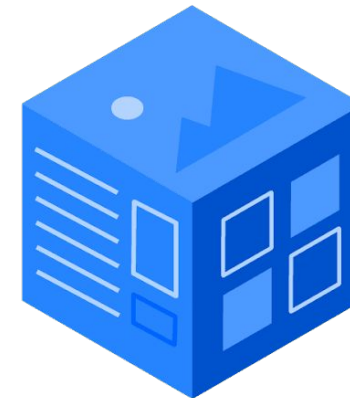


Using configs for AB testing

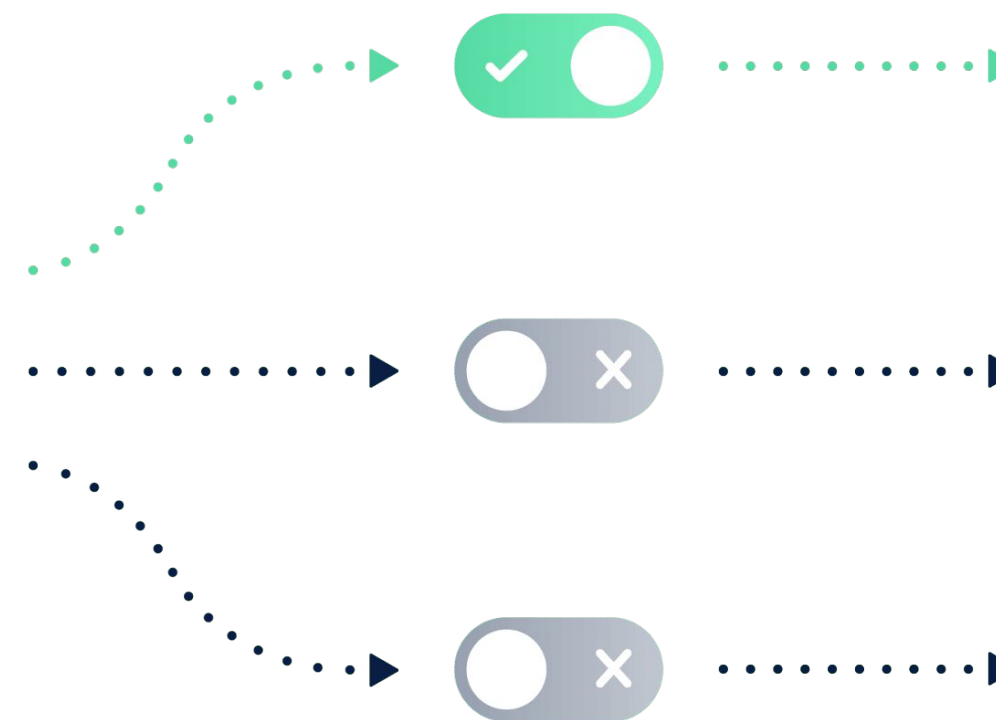
Using configs (e.g. when starting Docker container/pod) we can build infrastructure for AB testing or other needs.

Hydra is not suited for backend runtime (it's slow), but you may dump full config into .yaml file and give it to backend engineers.

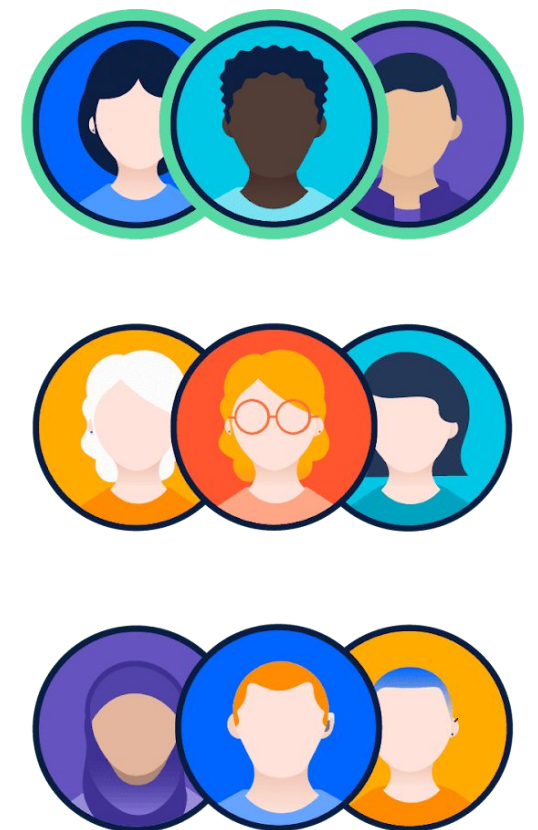
New feature



Feature Flags



Customers

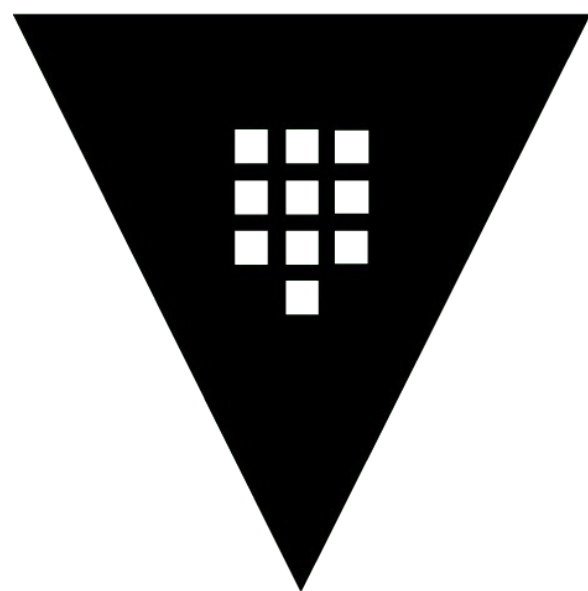


Special case - secrets

Local solution:

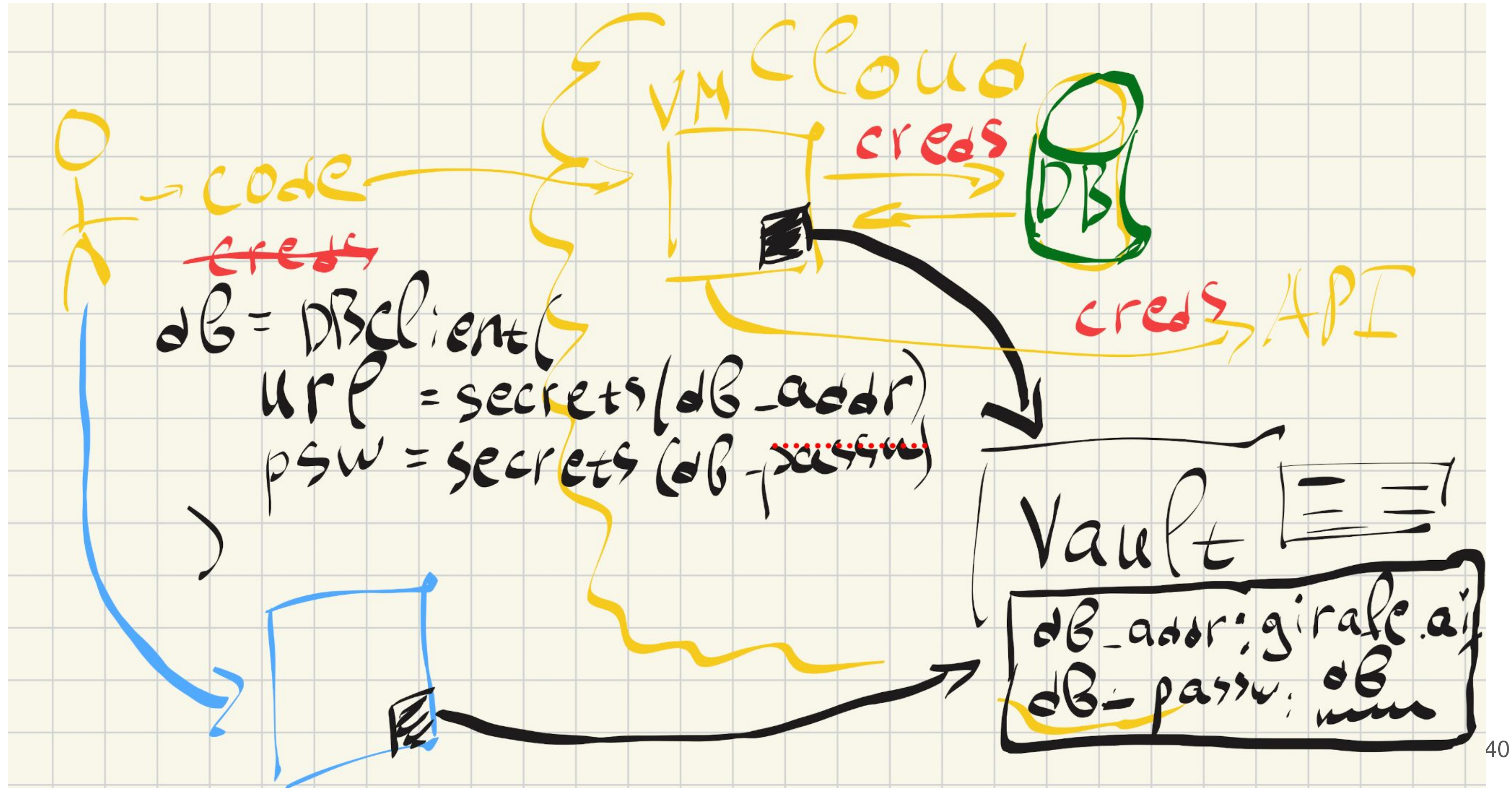
use [python-dotenv](#) with omegaconf [env resolver](#), [more info here](#)

Default industrial solution - Vault <https://github.com/hashicorp/vault>



HashiCorp
Vault

Secrets scheme



Local secrets

One widespread solution is dotenv library

<https://github.com/theskumar/python-dotenv>

```
from dotenv import load_dotenv
```

```
load_dotenv() # reads variables from a .env file and sets them in  
os.environ
```

```
from dotenv import dotenv_values
```

```
config = dotenv_values(".env")
```

```
# config = {"USER": "foo", "EMAIL": "foo@example.org"}
```

Colab Secrets

<https://x.com/GoogleColab/status/1719798406195867814>

<https://medium.com/@parthdasawant/how-to-use-secrets-in-google-colab-450c38e3ec75>

⋮

Secrets

🗑️

✕

🔍

Configure your code by storing environment variables, file paths, or keys. Values stored here are private, visible only to you and the notebooks that you select.

{x}

Secret name cannot contain spaces.

🔑

Notebook access

		Name	Value	Actions		
📁	☒	huggingface		👁️	📄	🗑️
	☐	kaggle		👁️	📄	🗑️
	☐	secretName	secretValue	👁️/	📄	🗑️

+ Add new secret

Access your secret keys in Python via:

📄

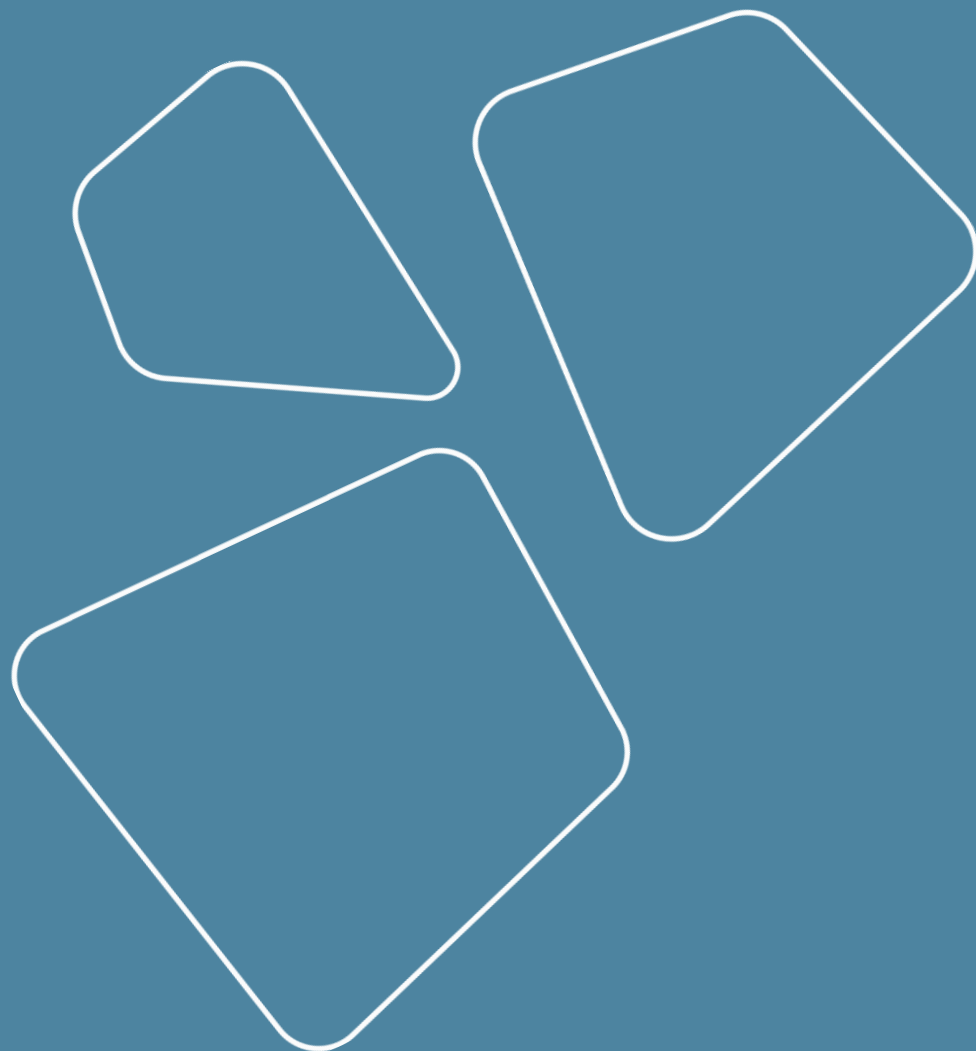
🗑️

```
from google.colab import userdata
userdata.get('secretName')
```

<>

Revise

1. Data sources
2. Data storage
 - a. small data
 - b. big data
3. Data engineering
 - a. ETL and ELT
 - b. datahub
4. Experiments parametrization
 - a. Hydra
 - b. Vault



Thanks for attention!

Questions?

